딥러닝 동적 자산배분

Motivation

□ 동기

- In a portfolio construction, the manager seeks to find the optimal trade-off among return, risk, and trading costs over several investment periods.
- Mostly, optimal decisions span over multi-periods, but solving high dimensional dynamic model is exceedingly difficult due to the curse of dimensionality, which makes it rarely used in practice.
- However, it is still valuable to develop the practical multi-period and high dimensional asset management tool which helps reduce future turnovers and transaction costs.
- Neural networks remarkably performs a model reduction, and approximate non-linear functions effectively. We apply a deep learning method developed by Maliar et al. (2021) into solving high dimensional dynamic asset allocation problem and compare its numerical accuracy and computational efficiency with the conventional approach.

Optimal dynamic rebalancing with transaction costs

- Rewards

- Garleanu and Pedersen. 2013 "Dynamic Trading with Predictable Returns and Transaction Costs, Journal of Finance"

  The model :

$$\max_{\{\omega_t\}_{t=0}^{\infty}} E_0\left[\sum_{t=0}^{\infty}\left(\frac{1}{1+r}\right)^t\left[-\frac{\Delta\omega_t'\Lambda\Delta\omega_t}{2}+\frac{1}{1+r}\left(\omega_t' r_{t+1}-\frac{a}{2}\omega_t'\Sigma\omega_t\right)\right]\right]$$

  subject to: $\omega_t \geq 0$ and $\sum_{s=1}^{S}\omega_{s,t} \leq 1$

  where ω_t is a portfolio at t , r_{t+1} is a stock price change, Σ is its variance, and $\frac{\Delta\omega_t'\Lambda\Delta\omega_t}{2}$ is a quadratic transaction cost.

- If one buys/sells $\Delta\omega_t = \omega_t - \omega_{t-1}$ shares, he/she immediately pays $\frac{\Delta\omega_t'\Lambda\Delta\omega_t}{2}$, expecting $\omega_t' r_{t+1} - \frac{a}{2}\omega_t'\Sigma\omega_t$ next period.

  Our goal is to find the optimal trading strategy $\{\omega_t\}_{t=0}^{\infty}$.

  Λ : Multi-dimensional Kyle's Lambda , a measure of market impact cost for an institutional investor.

Optimal dynamic rebalancing with transaction costs

- Signal process

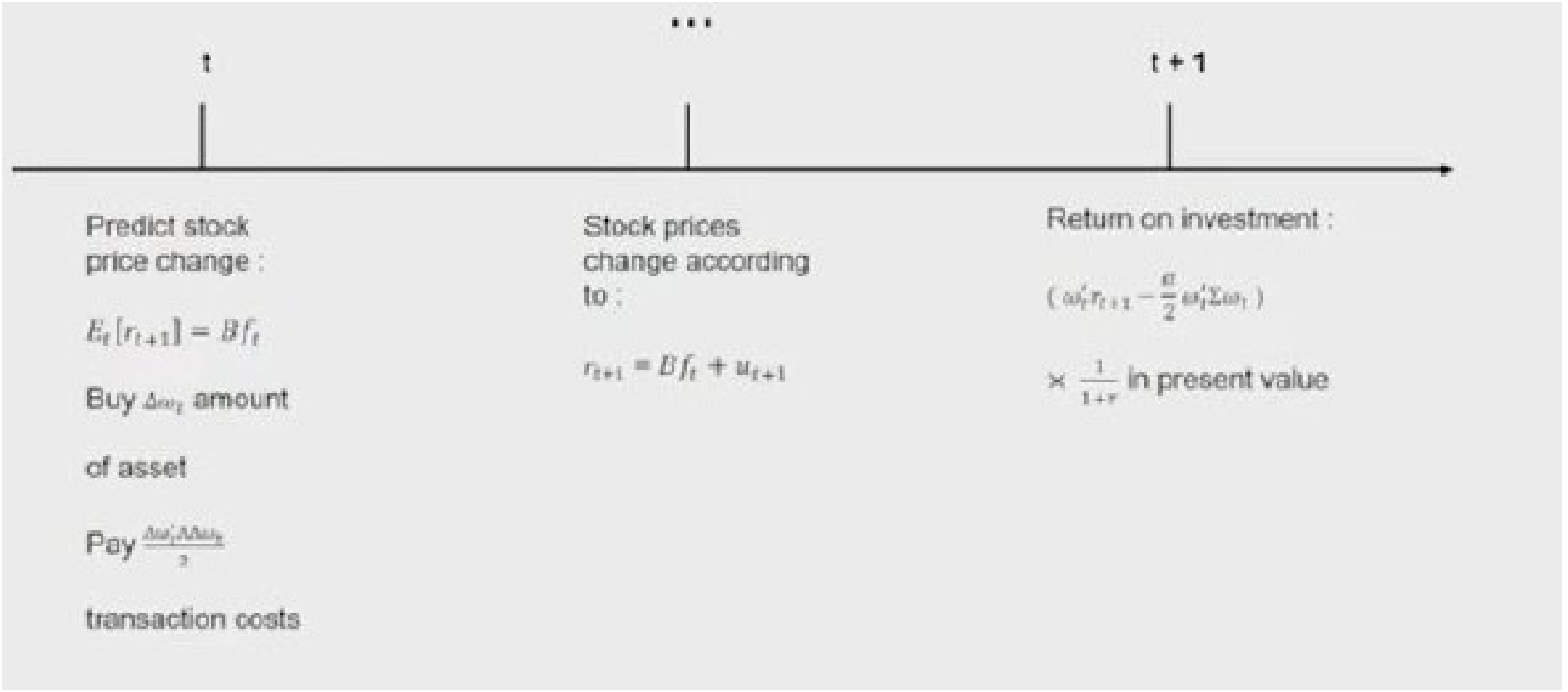
- Suppose stock price changes excess of risk-free return are predicted as :

$$r_{t+1} = p_{t+1} - (1 + r^f)p_t = \underbrace{B}_{\substack{factor \\ loadings}} f_t + u_{t+1}$$

$\Delta f_{t+1} = f_{t+1} - f_t = -\Phi f_t + \varepsilon_{t+1}$ where f_t is a vector of mean-reverting return predicting factors and u is a vector of white noise ($u \sim (0, \Sigma)$).

Optimal dynamic rebalancing with transaction costs

- Time line



High dimensional dynamic asset allocation

- Curse of dimensionality

- $V(\omega_{t-1}, f_t) = \max_{\omega_t} \left\{ -\frac{\Delta\omega_t' \Lambda \Delta\omega_t}{2} + \frac{1}{1+r}\left(\omega_t' E_t[r_{t+1}] - \frac{\alpha}{2}\omega_t' \Sigma \omega_t + E_t[V(\omega_t, f_{t+1})] \right) \right\}$

  subject to: $\omega_t \geq 0$ and $\sum_{s=1}^{S} \omega_{s,t} \leq 1$

  Solving High-dimensional dynamic programming (ex. value function / policy function iteration) becomes intractable because of exponential growth in complexity.

- If there are just 5 assets and 5 factors, then 10 grid points for each dimension amount to 10^{10} grids in total. (curse of dimensionality)

- One alternative is explicitly deriving optimality condition, casting it into non-linear regression equation, and apply a deep learning method

High dimensional dynamic asset allocation

- Curse of dimensionality

- Euler's equation (F.O.C) (for stationary problem) :

$$\underbrace{E[r'|\mathcal{F}] + \Lambda E[\Delta\omega'|\mathcal{F}]}_{(1)} - \underbrace{(\alpha\Sigma\omega + (1+r)\,\Lambda\Delta\omega)}_{(2)} + \eta - \eta_f\big(= E\big[\mathcal{R}^1_{S\times 1}|\mathcal{F}\big]\big) = 0$$

$$\eta \times \omega\,\big(= E\big[\mathcal{R}^2_{S\times 1}|\mathcal{F}\big]\big) = 0$$

$$\eta_f \times \big(1 - \textstyle\sum_{s=1}^{S}\omega_s\big)\,\big(= E[\mathcal{R}^3_{1\times 1}|\mathcal{F}]\big) = 0$$

The optimal portfolio equates marginal benefits (1) of rebalancing with marginal costs (2)

By solving the above system, one gets the policy function : $\omega = \varphi(\omega_{-1}, f)$.

High dimensional dynamic asset allocation

- Curse of dimensionality

- For a numerical approximation, parametrize the decision rule : $\omega = \varphi(\omega_{-1}, f, \theta)$ and substitute it into F.O.C :

$$E\left[\mathcal{R}\left(\omega_j, \varepsilon', u'; \varphi, \{\theta_i\}_{i=0}^{n}\right)\right] = 0 \text{ for } j = 1, 2, \ldots, n$$

so-called (orthogonal) 'collocation' imposes that the residuals $\mathcal{R}(\cdot)$ are exactly zero at each grid points $\{\omega_j\}_{j=0}^{n}$ on the continuous state space to get the estimate of parameters of polynomial approximating functions

- However, this approach becomes impractical too in high-dimensional problem

Deep learning methods for solving optimal dynamic rebalancing problems

- Deep dynamic rebalancing

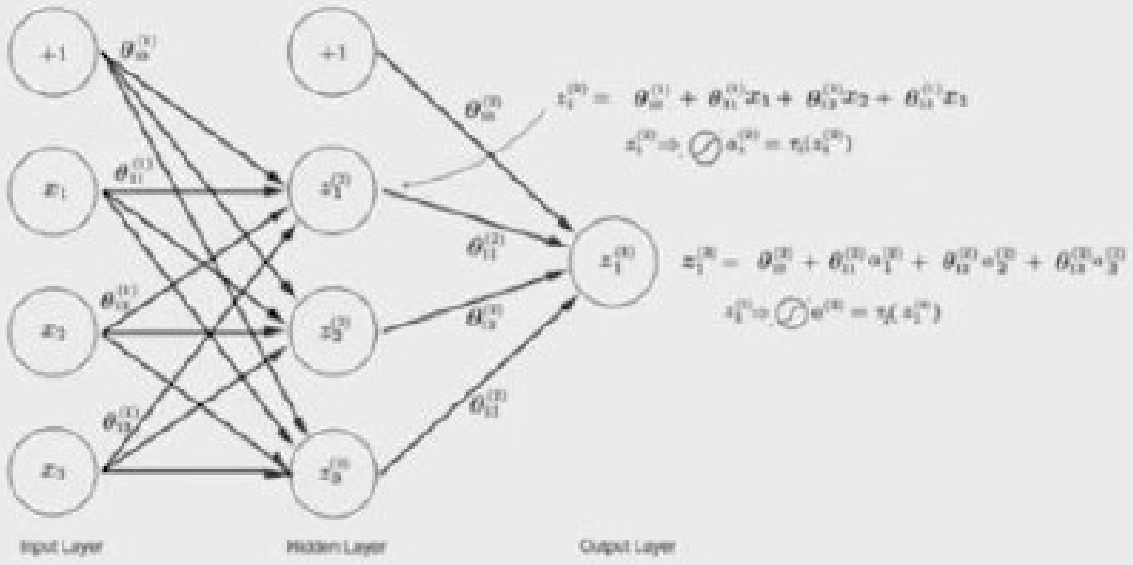


Fig 1. A neural network with one hidden layer

. Compared to the conventional polynomial approximating functions, neural networks are :

- The number of parameters grows linearly with dimensionality
- Highly flexible, well-suited for nonlinear problems
- Robust to multicollinearity & perform model reduction

Deep learning methods for solving optimal dynamic rebalancing problems

▢ Deep dynamic rebalancing

- The empirical risk is defined as a weighted square sum of the first order condition errors :

$$\Xi(\Theta) = E_{(\omega,f)}\left[\sum_{s=1}^{2S+1} v_s \left(E_{(\varepsilon,u)}[\mathcal{R}_s(\Theta, \varphi; \omega, \varepsilon', u')]\right)^2\right]$$

where $\{v_s\}_{s=1}^{S}$ is a weight (hyper parameters)

- AiO operator uses two independent batches (e_1, e_2) which mimic the original random shock e in the sense that

$$(E_e[f(e)])^2 \cong E_{(e_1,e_2)}[f(e_1)f(e_2)] = E_{e_1}[f(e_1)]E_{e_2}[f(e_2)]$$

- We simulate the model to produce data $\{\omega_j\}_{j=1}^{N}$, and train our multilayer neural network by using ADAM

Numerical Example

□ 수치 예시

Variable	Values	Description
α	0.5	Absolute risk-aversion parameter
r	0.0563	Time-preference rate
λ	3	Transaction cost parameter
ϕ	0.084	Mean-reversion coefficients (factors) ($\Phi = diag(\phi, \ldots, \phi)$)
B	$I_{5\times5}$	Factor loading

Deep learning methods for solving optimal dynamic rebalancing problems

□ 결과

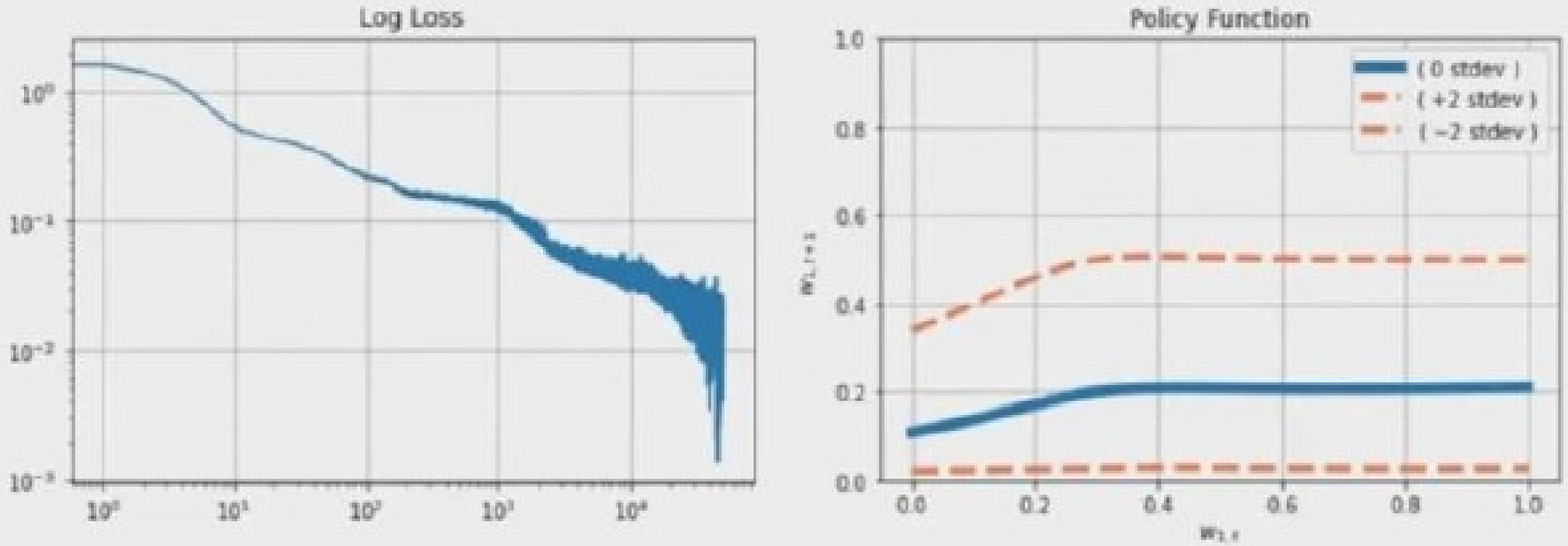
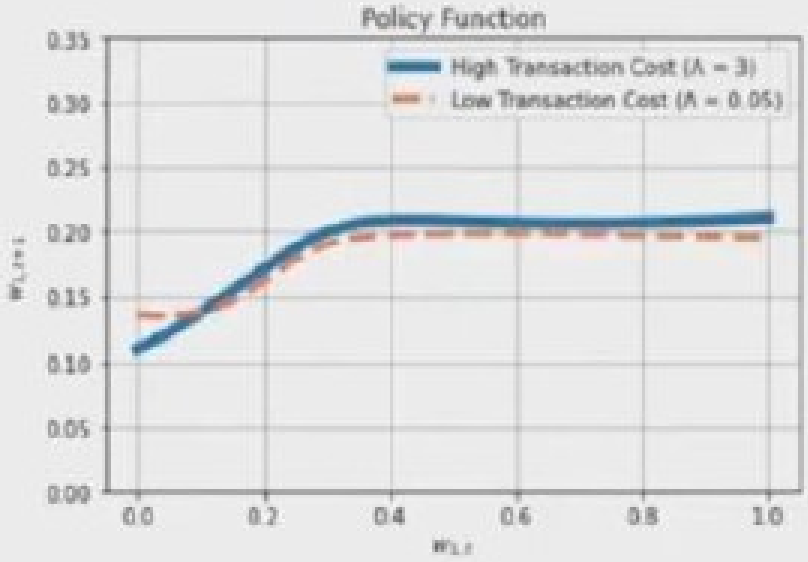


Fig 2. ±2 standard deviation changes in return predicting factor 1

Deep learning methods for solving optimal dynamic rebalancing problems

□ 결과



$$\underbrace{E[r'|\mathcal{F}] + \Lambda E[\Delta\omega'|\mathcal{F}]}_{(3)} - \underbrace{(a\Sigma\omega + (1+r)\,\Lambda\Delta\omega)}_{(2)} + \eta - \eta_f = 0$$

Higher transaction costs ($\lambda = 0.05 \rightarrow 3$) lead to slower rebalancing in portfolio

(Marginal cost of rebalancing gets higher)

Fig 3. Increase in transaction cost ($\lambda = 0.05 \rightarrow 3$)

Deep learning methods for solving optimal dynamic rebalancing problems

▢ 결론

- Deep learning approach can be used as alternative approach to solving high-dimensional and non-linear dynamic optimization

  Neural networks yield a flexible approximation to highly non-linear functions using, for example, ReLu and Sigmoid.

- Especially, it converts a high-dimensional state space more compact and user-defined hidden layers of neurons

- Future extension can be 1). Introduction of linear cost as well as quadratic form, 2). Dynamic asset allocation with model uncertainty